XCPC 通用杂项板子

本文件只放不属于现有图论、数论、数据结构和 DP 文件，但能独立复用的高频模块。默认 C++17；变量名保持短，代码块内只写接口和参数，完整前置条件放在代码块外。

通用提交骨架见同目录的 `火车头.cpp`。火车头只保留每题都可能用到且无运行时负担的内容；PBDS、Miller-Rabin、计算几何类型和大数组按题复制，不放进公共骨架。

0. 选型

需求	模块	复杂度
极限整数输入输出	缓冲 FastIO	均摊 `O(字符数)`
单调判定边界	整数/实数二分	`O(log 范围)`
单峰函数极值	三分	固定迭代或 `O(log 范围)`
`n` 约 40 的子集问题	折半搜索 MITM	`O(2^(n/2) log 2^(n/2))`
每一列恰好覆盖一次	DLX 精确覆盖	指数级，剪枝通常很强

1. 缓冲输入输出

普通 `cin/cout` 关闭同步并解除绑定后足够快时，优先使用标准流。只有输入输出达到数百万个整数并确认卡常时，再换本段。

`FastIO io` 建议声明为全局对象，避免两个 `1 MiB` 缓冲区占用主线程栈。`read(x)` 支持有符号和无符号整数，成功返回 `true`，读到 EOF 返回 `false`；`write(x,end)` 输出整数和结尾字符。不要和 `cin/cout/scanf/printf` 混用，否则缓冲顺序不可控。

```
struct FastIO {
    static const int S = 1 << 20;
    int ip = 0, in = 0, op = 0;
    char ib[S], ob[S];

    ~FastIO() { flush(); }
    char gc() { // 返回下一字节；EOF 返回 0
        if (ip == in) in = fread(ib, 1, S, stdin), ip = 0;
        return ip == in ? 0 : ib[ip++];
    }
    void pc(char c) { // c：待输出字符
        if (op == S) flush();
        ob[op++] = c;
    }
    void flush() { // 写出当前输出缓冲
        if (op) fwrite(ob, 1, op, stdout), op = 0;
    }

    template<class T> bool read(T &x) { // x：接收整数
```

```
        static_assert(is_integral_v<T>);
        char c = gc();
        while (c && c <= ' ') c = gc();
        if (!c) return false;
        bool neg = c == '-';
        if (neg || c == '+') c = gc();
        using U = make_unsigned_t<T>;
        U v = 0;
        while (c >= '0' && c <= '9') v = v * 10 + (c - '0'), c = gc();
        x = neg ? (T)(U(0) - v) : (T)v;
        return true;
    }

    bool read(string &s) { // s：接收一个非空白字符串
        char c = gc();
        while (c && c <= ' ') c = gc();
        if (!c) return false;
        s.clear();
        while (c > ' ') s.push_back(c), c = gc();
        return true;
    }

    template<class T> void write(T x, char end = '\n') {
        // x：待输出整数；end：结尾字符，传 0 表示不输出结尾
        static_assert(is_integral_v<T>);
        using U = make_unsigned_t<T>;
        U v;
        if constexpr (is_signed_v<T>) {
            if (x < 0) pc('-'), v = U(0) - (U)x;
            else v = (U)x;
        } else v = x;
        char s[32]; int n = 0;
        do s[n++] = char('0' + v % 10), v /= 10; while (v);
        while (n) pc(s[--n]);
        if (end) pc(end);
    }
};
```

本模板不直接支持 `__int128`，因为标准类型萃取对它的支持依编译器而异；高精度输入输出使用“数字数据结构板子”中的独立函数。

2. 二分与三分

整数二分采用闭区间：`first_true(l,r,ok)` 要求区间内至少一个 `true`，且判定从假变真；`last_true` 要求从真变假。若答案可能不存在，应先在代码外检查端点。

实数二分和三分固定迭代次数，不依赖 `eps` 退出，能避免精度导致的死循环。`real_first` 要求 `ok(x)` 单调从假变真；`ternary_min` 要求函数先降后升。整数三分最后暴力检查不超过 7 个点。

```
template<class F> long long first_true(long long l, long long r, F ok) {
    // l、r：闭区间；ok：从假变真的判定
```

```
    while (l < r) {
        long long m = l + (r - l) / 2;
        if (ok(m)) r = m;
        else l = m + 1;
    }
    return l;
}

template<class F> long long last_true(long long l, long long r, F ok) {
    // l、r：闭区间；ok：从真变假的判定
    while (l < r) {
        long long m = l + (r - l + 1) / 2;
        if (ok(m)) l = m;
        else r = m - 1;
    }
    return l;
}

template<class F> long double real_first(long double l, long double r, F ok) {
    // l、r：答案所在区间；ok：从假变真的判定
    for (int i = 0; i < 100; i++) {
        long double m = (l + r) / 2;
        if (ok(m)) r = m;
        else l = m;
    }
    return (l + r) / 2;
}

template<class F> long double ternary_min(long double l, long double r, F f) {
    // l、r：搜索区间；f：先降后升的实函数
    for (int i = 0; i < 150; i++) {
        long double x = (2 * l + r) / 3, y = (l + 2 * r) / 3;
        if (f(x) <= f(y)) r = y;
        else l = x;
    }
    return (l + r) / 2;
}

template<class F> long long ternary_int(long long l, long long r, F f) {
    // l、r：闭区间；f：先降后升的整数函数；返回最小值位置
    while (r - l > 6) {
        long long x = l + (r - l) / 3, y = r - (r - l) / 3;
        if (f(x) <= f(y)) r = y - 1;
        else l = x + 1;
    }
    long long p = l;
    for (long long x = l + 1; x <= r; x++) if (f(x) < f(p)) p = x;
    return p;
}
```

三分只适用于单峰函数。离散函数存在平台段时，上面返回平台中的某个最优点；若要最左或最右最优点，得到最优值后再二分边界。

3. 折半搜索 Meet-in-the-Middle

当 `n` 在 35 到 46 左右、`2^n` 无法承受时，把元素分成两半分别枚举子集和。

- `mitm_max(a,lim)`：返回不超过 `lim` 的最大子集和；不存在时返回 `LLONG_MIN`。元素允许为负数。
- `mitm_count(a,L,R)`：返回子集和落在闭区间 `[L,R]` 的方案数。

时间 $O(2^{n/2}\log 2^{n/2})$，空间 $O(2^{n/2})$，通常适合 `n<=44`。要求全部子集和、`lim-x` 等中间运算能放进 `long long`。

```
vector<long long> sub_sums(const vector<long long> &a, int l, int r) {
    // a：原数组；[l,r)：枚举范围
    vector<long long> s{0};
    for (int i = l; i < r; i++) {
        int n = s.size();
        s.reserve(2 * n);
        for (int j = 0; j < n; j++) s.push_back(s[j] + a[i]);
    }
    return s;
}

long long mitm_max(const vector<long long> &a, long long lim) {
    // a：元素；lim：子集和上界
    int m = a.size() / 2;
    auto x = sub_sums(a, 0, m), y = sub_sums(a, m, a.size());
    sort(y.begin(), y.end());
    long long ans = LLONG_MIN;
    for (long long v : x) {
        auto it = upper_bound(y.begin(), y.end(), lim - v);
        if (it != y.begin()) ans = max(ans, v + *prev(it));
    }
    return ans;
}

long long mitm_count(const vector<long long> &a, long long L, long long R) {
    // a：元素；L、R：子集和闭区间
    int m = a.size() / 2;
    auto x = sub_sums(a, 0, m), y = sub_sums(a, m, a.size());
    sort(y.begin(), y.end());
    auto count_le = [&](long long z) {
        long long r = 0;
        for (long long v : x) r += upper_bound(y.begin(), y.end(), z - v) -
y.begin();
        return r;
    };
    return count_le(R) - (L == LLONG_MIN ? 0 : count_le(L - 1));
}
```

若只问是否存在某个和，可以对一半排序后逐项二分；若所有数非负且目标值不大，普通 `bitset` 背包通常更快。

4. DLX：精确覆盖

精确覆盖要求从若干行中选一些，使每一列恰好被 `1` 个所选行覆盖。典型应用是数独、棋盘放置和集合精确划分。

`DLX d(m)` 建立 `m` 列；`add_row(id,cs)` 加入编号为 `id` 的一行，`cs` 是该行含 `1` 的列号，范围 `1..m` 且不能重复；`solve(ans)` 找到任意一组解并把行号写入 `ans`，无解返回 `false`。同一对象可以在结构不变时重复调用 `solve`。

最坏复杂度仍为指数级，但选择当前元素最少的列后通常很快。下面只实现"每列恰好一次"的 Algorithm X；"至少一次"或"至多一次"不是同一个模型，不能直接套。

```
struct DLX {
    int m;
    vector<int> l, r, u, d, c, s, row;

    DLX(int m): m(m), l(m + 1), r(m + 1), u(m + 1), d(m + 1),
                c(m + 1), s(m + 1), row(m + 1) {
        for (int i = 0; i <= m; i++)
            l[i] = i - 1, r[i] = i + 1, u[i] = d[i] = c[i] = i;
        l[0] = m, r[m] = 0;
    }

    void add_row(int id, const vector<int> &cs) { // id：行号；cs：本行覆盖列
        int h = 0;
        for (int x : cs) {
            assert(1 <= x && x <= m);
            int z = l.size();
            l.push_back(z), r.push_back(z), u.push_back(u[x]), d.push_back(x);
            c.push_back(x), s.push_back(0), row.push_back(id);
            d[u[x]] = z, u[x] = z, s[x]++;
            if (!h) h = z;
            else {
                l[z] = l[h], r[z] = h;
                r[l[h]] = z, l[h] = z;
            }
        }
    }

    void cover(int x) { // x：待删除列
        l[r[x]] = l[x], r[l[x]] = r[x];
        for (int i = d[x]; i != x; i = d[i])
            for (int j = r[i]; j != i; j = r[j])
                u[d[j]] = u[j], d[u[j]] = d[j], s[c[j]]--;
    }
    void uncover(int x) { // x：恢复列，顺序必须与 cover 相反
        for (int i = u[x]; i != x; i = u[i])
            for (int j = l[i]; j != i; j = l[j])
                s[c[j]]++, u[d[j]] = j, d[u[j]] = j;
        l[r[x]] = x, r[l[x]] = x;
    }
```

```
    bool dfs(vector<int> &ans) {
        if (!r[0]) return true;
        int x = r[0];
        for (int j = r[x]; j; j = r[j]) if (s[j] < s[x]) x = j;
        if (!s[x]) return false;
        cover(x);
        for (int i = d[x]; i != x; i = d[i]) {
            ans.push_back(row[i]);
            for (int j = r[i]; j != i; j = r[j]) cover(c[j]);
            if (dfs(ans)) {
                for (int j = l[i]; j != i; j = l[j]) uncover(c[j]);
                uncover(x);
                return true;
            }
            for (int j = l[i]; j != i; j = l[j]) uncover(c[j]);
            ans.pop_back();
        }
        uncover(x);
        return false;
    }

    bool solve(vector<int> &ans) { // ans：接收一组所选行号
        ans.clear();
        return dfs(ans);
    }
};
```

数独建模时，每个候选 `(行,列,数字)` 是一行，约束列分别表示"格子被填一次、每行每数一次、每列每数一次、每宫每数一次"，共 `4N^2` 列。

5. 未加入的伪通用模板

- 模拟退火、随机爬山的核心在题目专用评价函数、扰动方式和温度参数，固定代码通常没有可复用价值。
- 通用插头 DP 的状态编码和转移与网格连通性定义强绑定；只保留"动态规划板子"中的轮廓线基础模型。
- 并行二分、整体二分已经在"数据结构板子"给出使用条件，实际修改/查询操作应按题目编写。